Distributed Task Queue and Job Scheduler

Priority jobs, scheduled execution, retries, worker-style processing, dead-letter design

Project Type	Best For	Deliverables
Backend / Distributed Systems Demo	Amazon SDE I resume + interview explanation	Runnable code, demo script, tests, PDFs

1. Problem Statement

Backend platforms often need to execute slow or delayed jobs outside the request-response path, while still handling failure, retry, and priority.

2. Architecture Flow

Client -> Job API -> Scheduler -> Priority Queue -> Worker Runner -> Retry / Dead Letter Logic -> Job Store

3. Core Features

- Submit jobs with priority and schedule time
- Worker-style processing through run-next endpoint
- Retry handling for temporary failures
- Dead-letter queue concept for permanently failed jobs
- Observable job events and status counters
- Automated tests covering retry and completion

4. API Surface

Endpoint	Purpose
POST /jobs	Create a job
GET /jobs	List jobs and recent events
POST /workers/run-next	Process the next due high-priority job
GET /stats	Return status counts
POST /reset	Reset demo state

5. How to Run the Demo

Run these commands from the project folder:

```
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
pytest -q
```

6. Demo Talking Points

I designed this to show how background execution works. A request creates a job quickly, and workers process it independently using priority, retries, and status transitions.

7. Resume Bullet

> Developed a distributed job scheduling system with priority queues, scheduled execution, retry handling, worker processing, event tracking, and automated test coverage.

8. What to Say in Interview

- Why I built it: to practice backend workflows that need correctness, reliability and scalability.
- Main challenge: keeping state transitions correct when retries or failures happen.
- What I learned: data structures, APIs, testing, failure handling, and clean modular design.
- Future scale-up: replace in-memory components with Redis/Kafka/SQS, add auth, metrics, tracing and cloud deployment.

9. Demo Output Snapshot

POST /jobs normal


```
{
"id": "d4d68771",
"name": "image-resize",
"payload": {},
"priority": 6,
"scheduled_at": 1787678957.7653143,
"max_retries": 2,
"attempts": 0,
"fail_first": false,
"status": "PENDING",
"result": null
}
```


POST /jobs fail-first


```
{
"id": "5024cb66",
"name": "email-retry",
"payload": {},
"priority": 10,
"scheduled_at": 1787678957.7667933,
"max_retries": 2,
"attempts": 0,
"fail_first": true,
"status": "PENDING",
"result": null
}
```

POST /workers/run-next


```
{
"id": "5024cb66",
"name": "email-retry",
"payload": {},
"priority": 10,
"scheduled_at": 1787678958.7683692,
"max_retries": 2,
"attempts": 1,
"fail_first": true,
"status": "RETRY",
"result": "temporary provider failure"
}
```